Linear Search

1. Linear Search.

```
static int linearSearch(int[] arr, int target) {
        if (arr.length == 0) {
            return -1;
        }
        for (int i=0; i < arr.length; i++) {
            int ele = arr[i];
            if (ele == target) {
                return i;
            }
        }
       return -1;
    }
```

2. Search in String.

```
static boolean SearchInString (String str, char target){
        // if (str.length() == 0) {
        //     return false;
        // }
        // for (int i=0; i < str.length(); i++) {
        //     if (target == str.charAt(i)) {
        //         return true;
        //     }
        // }
        // return false;
        for (char ch : str.toCharArray()) {
            if (ch == target) {
                return true;
            }
        }
        return false;
    }
```

3. Search in Range.

```
 static int SearchInRange (int[] arr, int target, int start, int
end) {
        if (arr.length == 0) {
            return -1;
        }
        for (int index = start; index <= end; index++) {
            int ele = arr[index];
            if (ele == target) {
                return index;
            }
        }
        return -1;
    }
```

4. Minimum Value. (2D Array)

```
static int[][] findMin (int[][] arr, int target) {
        for ( int i=0; i < arr.length; i++) {
            for (int j=0; j<arr[i].length; j++) {
                if (arr[i][j] == target) {
                    return new int[][]{{i, j}};
                }
            }
        }
        return new int[][]{{-1, -1}};
    }
```

5. Maximum Value. (2D Array)

```
static int findMax(int[][] arr) {
    int max = Integer.MIN_VALUE;

    for (int i = 0; i < arr.length; i++) {
        for (int j = 0; j < arr[i].length; j++) {
            if (arr[i][j] > max) {
                max = arr[i][j];
            }
        }
    }

    return max;
}
```

6. **Find Numbers.**

```
static int findNumbers(int[] nums) {
        int count = 0;

        for (int num : nums) {
            if (even(num)) {
                count++;
            }
        }
        return count;
    }
```

7. **Even Digits.**

```
static boolean even(int num) {
        int numberOfDigits = digits(num);
        return numberOfDigits % 2 == 0;
    }
```

8. **Number of Digits.**

```
static int digits(int num) {
        if (num < 0) {
            num = num * -1;
        }

        if (num == 0) {
            return 1;
        }
        return (int) (Math.log10(num)) + 1;
    }
```

9. Maximum Wealth.

```java
// Leetcode : 1672
// https://leetcode.com/problems/richest-customer-
wealth/description/
class Solution {
    public int maximumWealth(int[][] accounts) {
        int ans = Integer.MIN_VALUE;
        for(int person = 0; person < accounts.length; person++){
            int sum = 0;
            for(int account = 0; account<
accounts[person].length; account++){
                sum += accounts[person][account];
            }
            if(sum>ans){
                ans = sum;
            }
        }
        return ans;
    }

    public static void main(String[] args) {

        Solution obj = new Solution();

        int[][] accounts = {
                {1, 2, 3},
                {3, 2, 1},
                {4, 5, 6}
        };

        int result = obj.maximumWealth(accounts);

        System.out.println("Maximum Wealth: " + result);
    }
}
```